



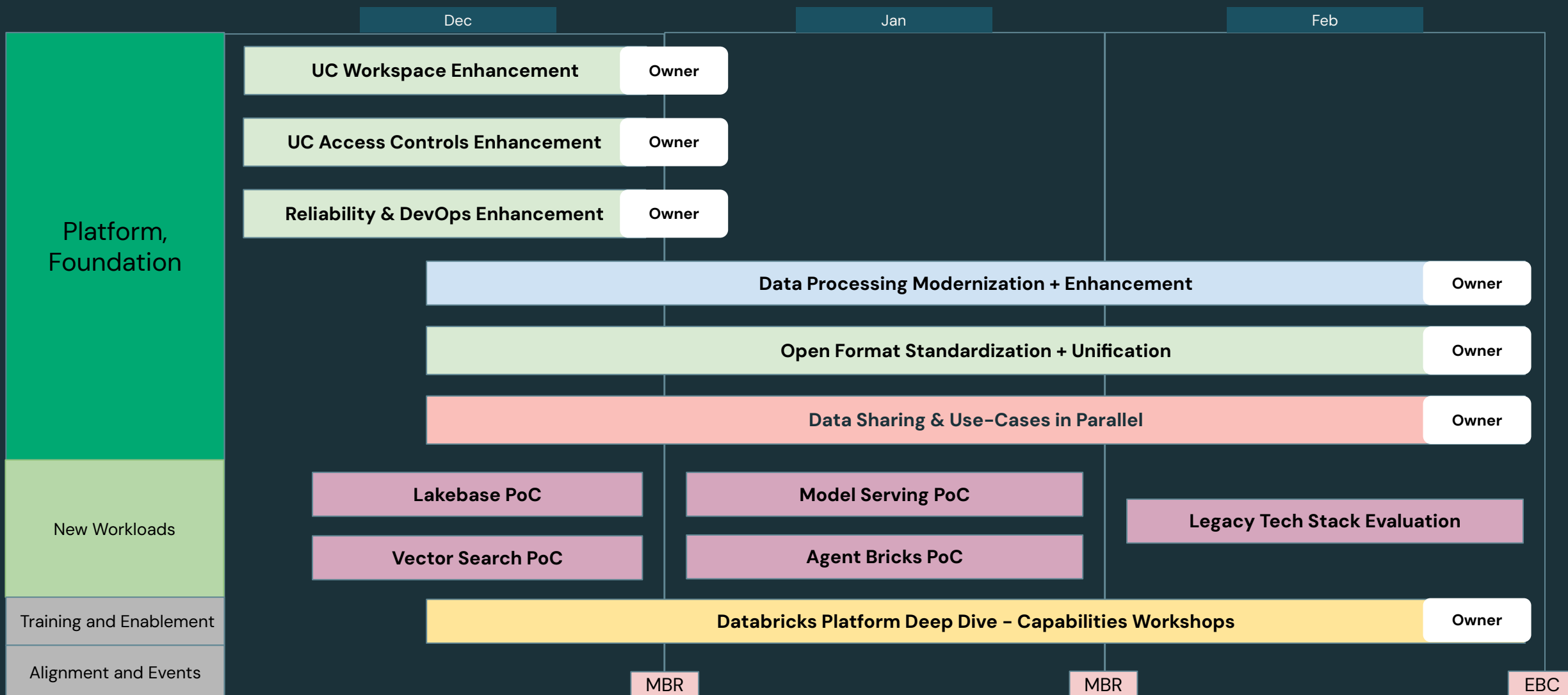
Unity Catalog

Best Practices & Rapid Start





Rollout Plan – Execution & Ownership



Immediate Focus Areas (next 30 days)

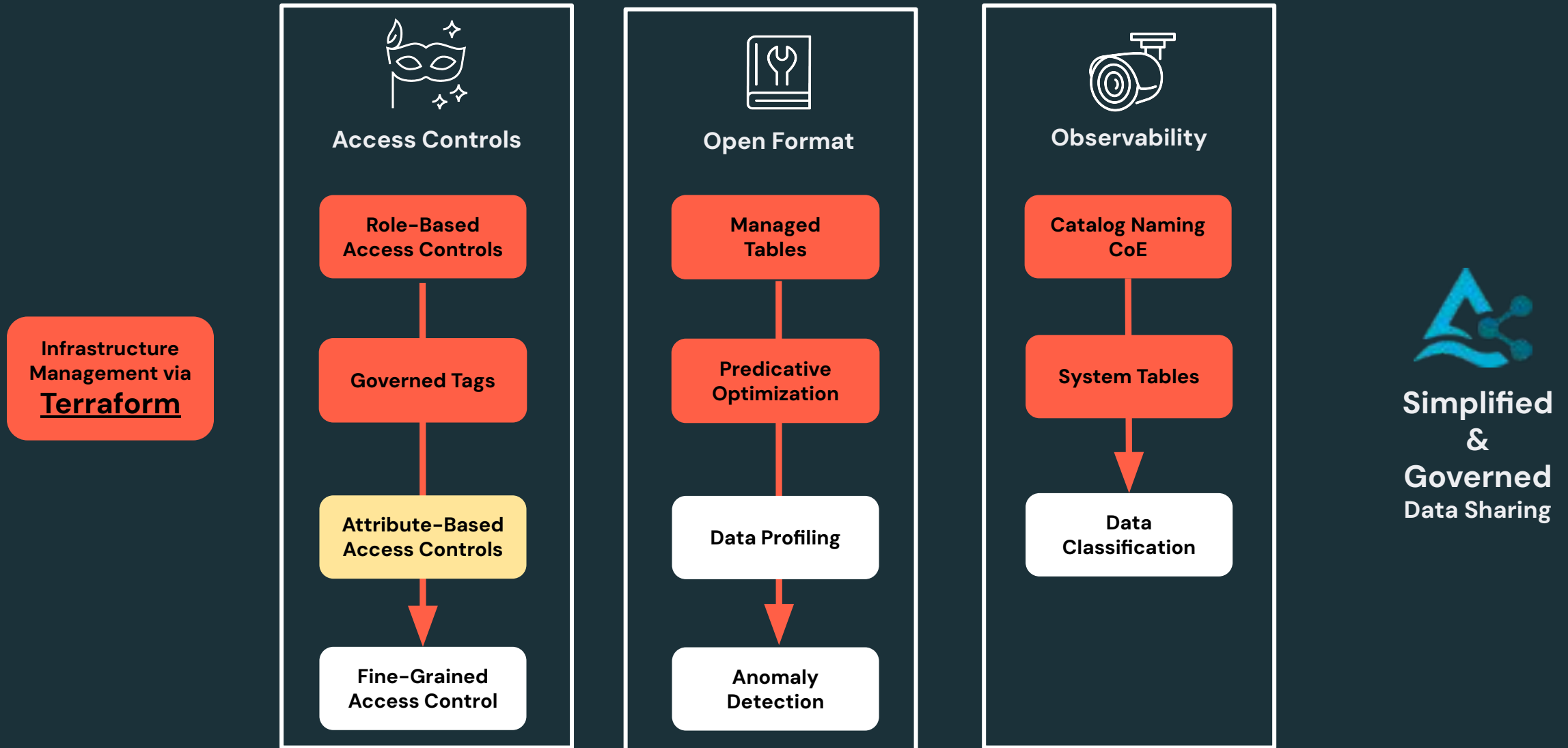
→ Decisions

- ◆ Publishing plan (centralized, distributed)
- ◆ Naming convention for catalog schema design

→ Actions

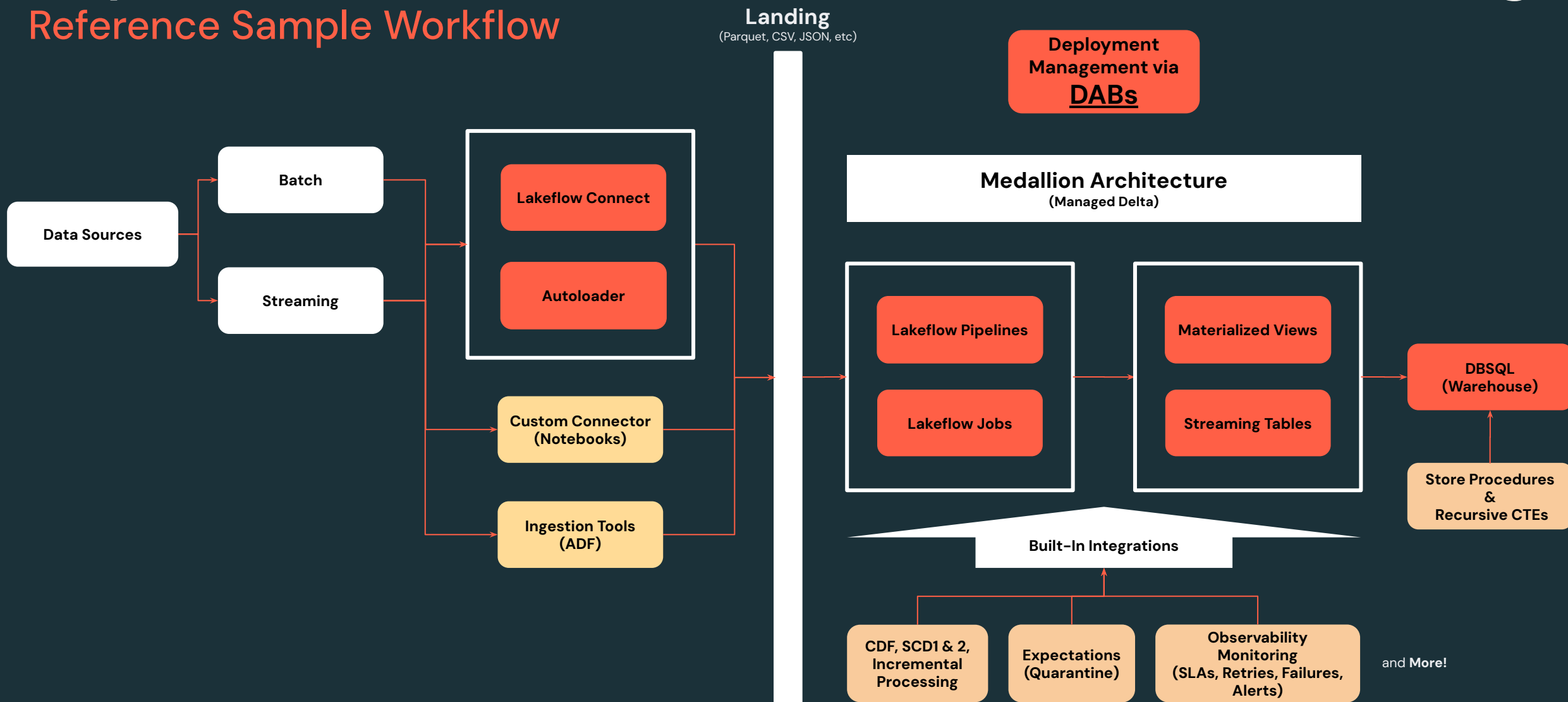
- ◆ Roll-out Catalog Design
- ◆ Access Control
- ◆ Tagging
- ◆ Runtime upgrade and runtime release process

Data Governance Implementation Architecture

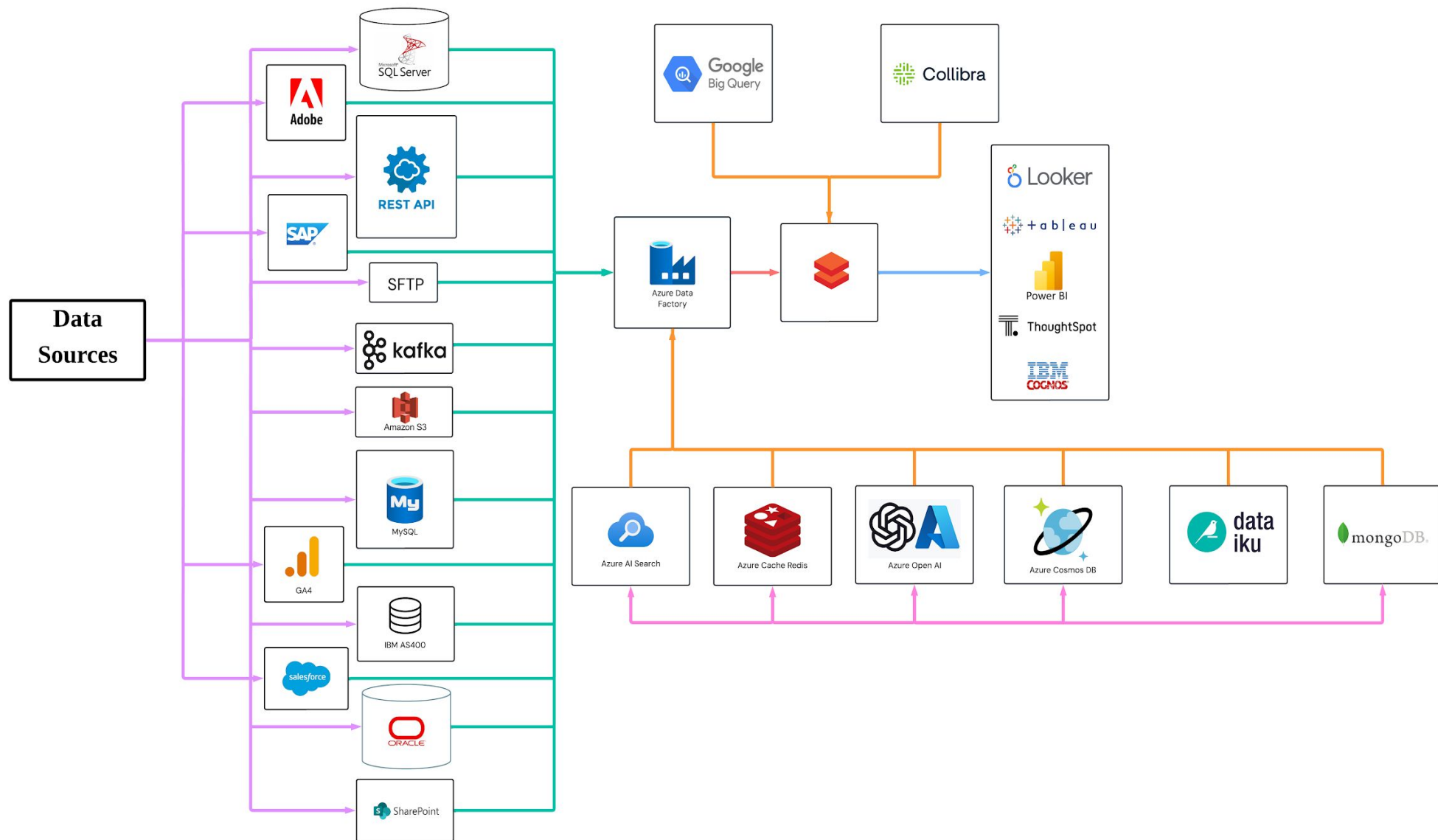


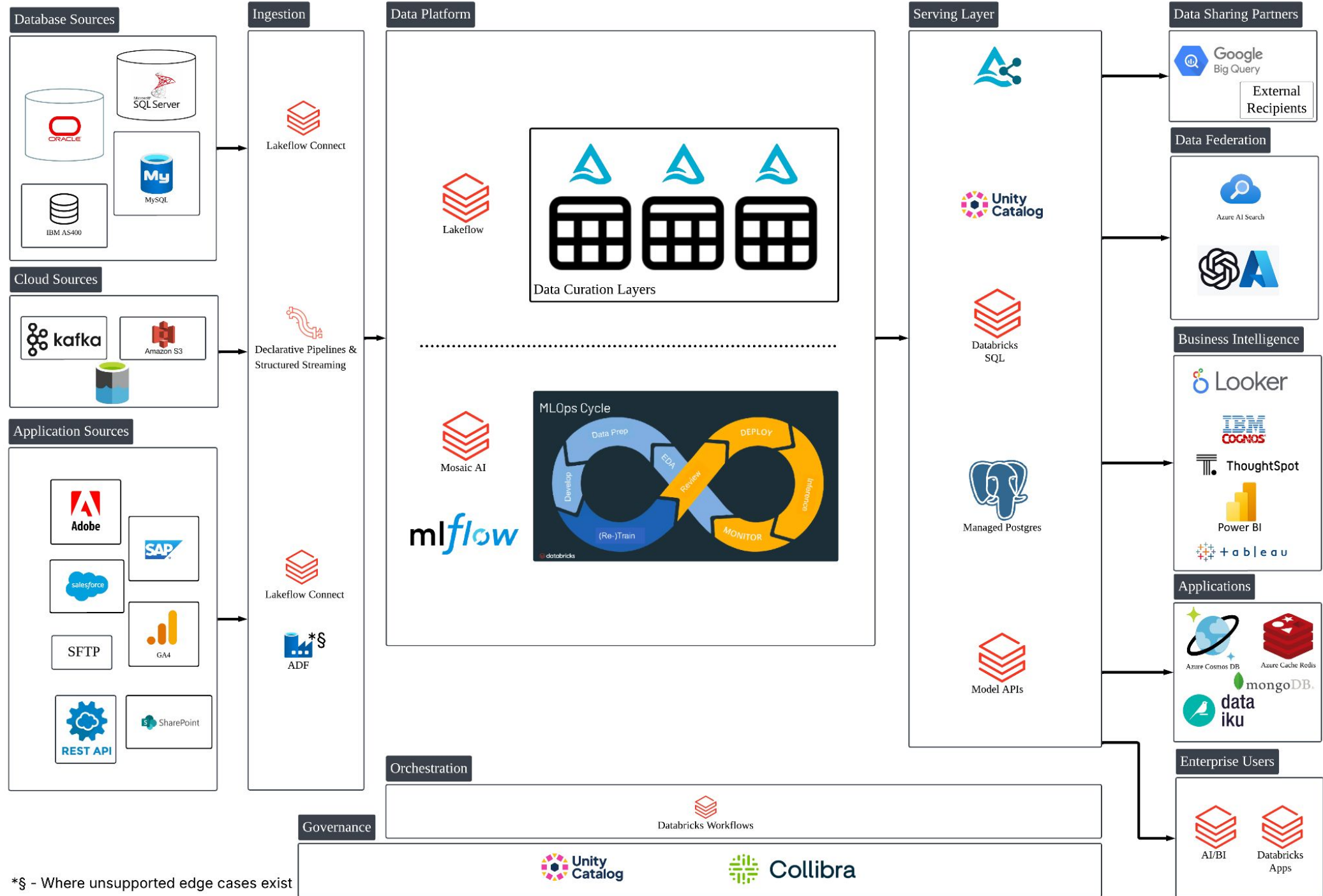
Implementation Architecture – Data Processing

Reference Sample Workflow



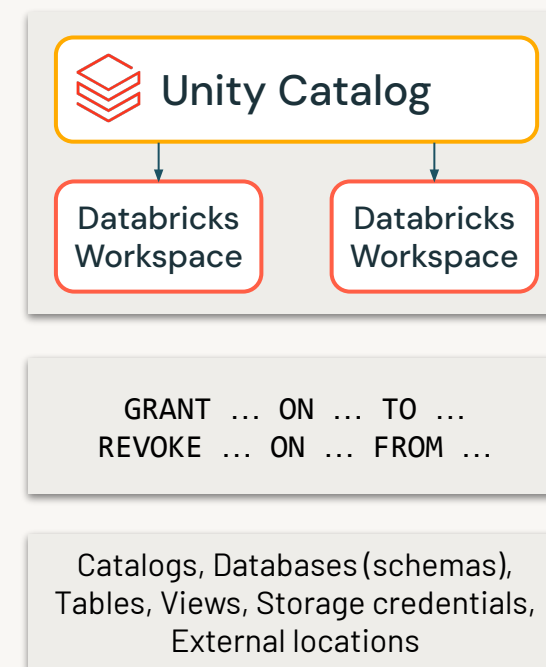
Current State Architecture





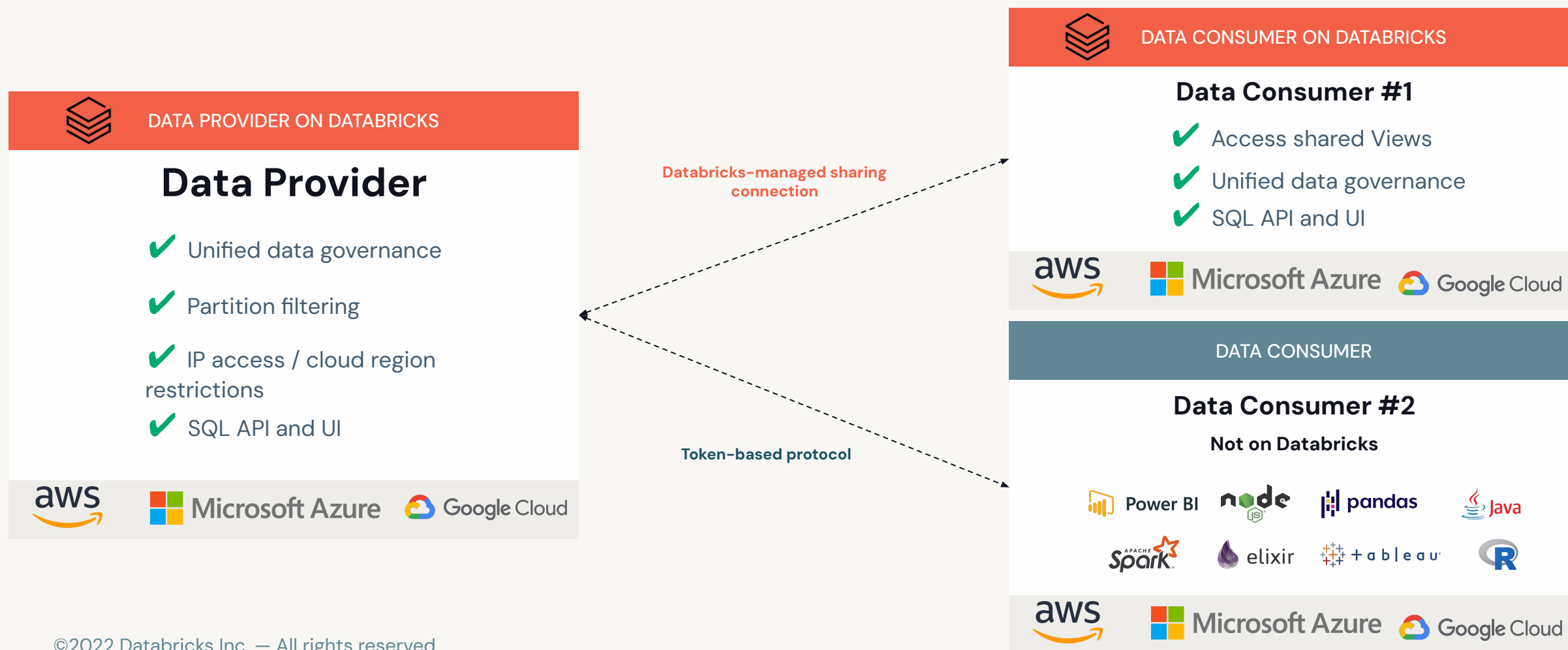
Unity Catalog – Key Capabilities

- Centralized metadata and user management
- Centralized data access controls
- Data automatic lineage
- Data access auditing
- Data search and discovery
- Secure data sharing with Delta Sharing
- System Tables
- Marketplace



Delta Sharing

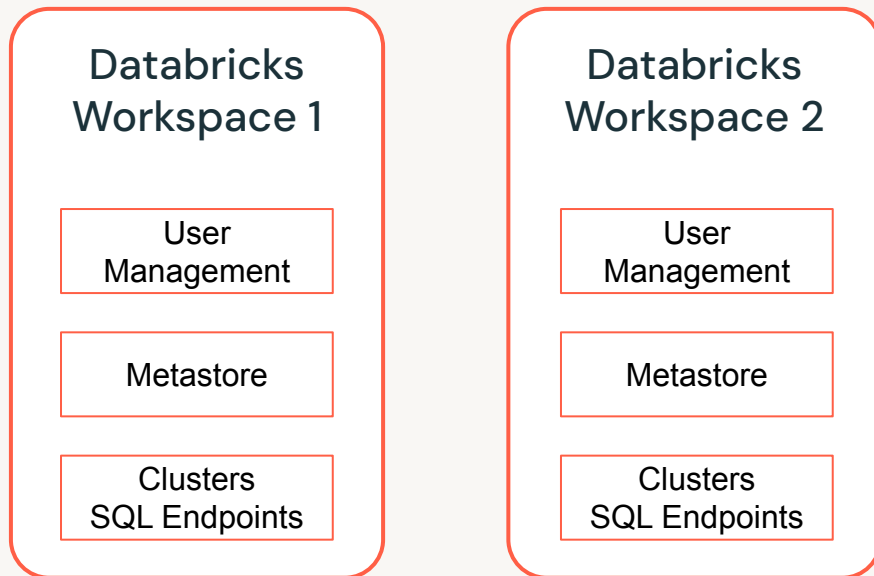
Secure sharing of data assets



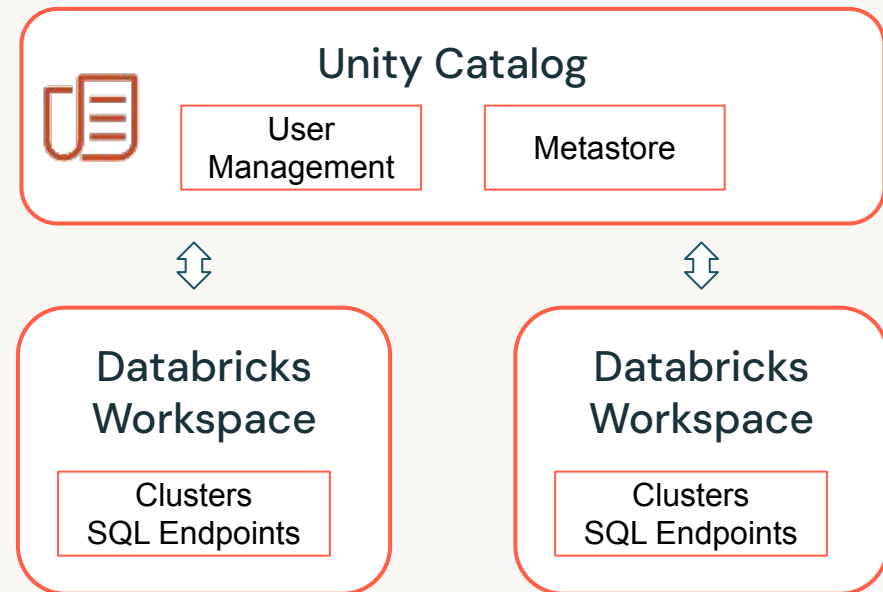
Centralized Metadata and User Management

Create a unified view of your data estate

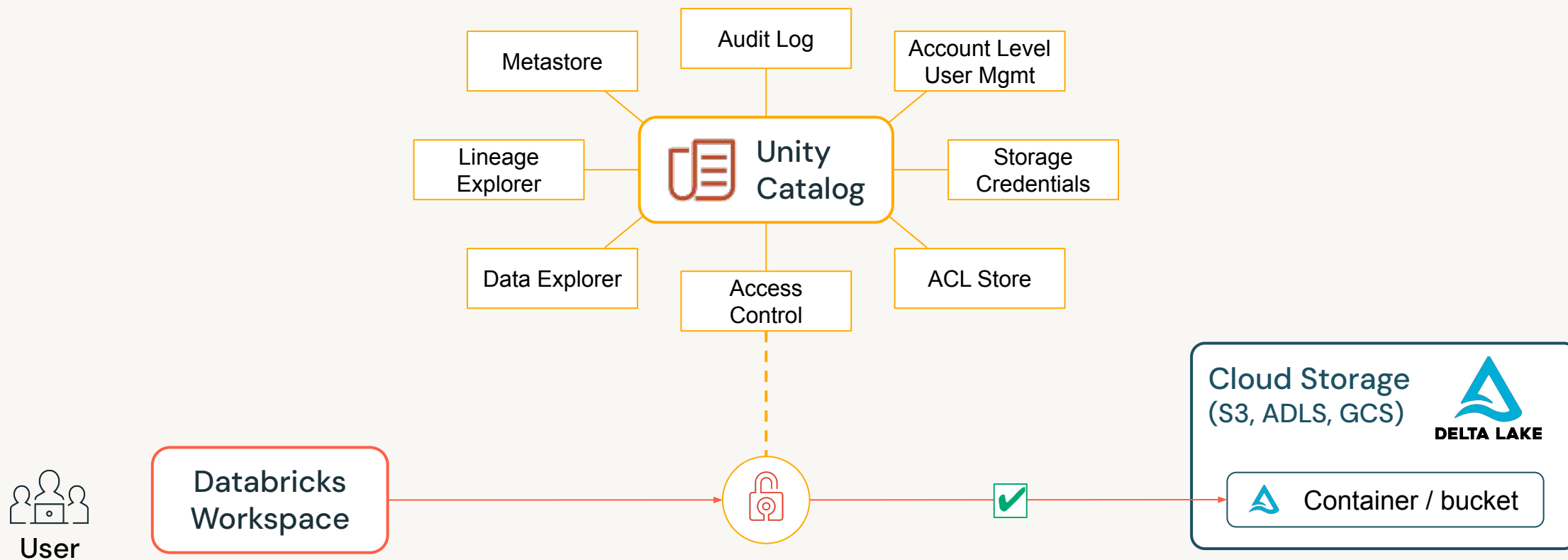
Without Unity Catalog



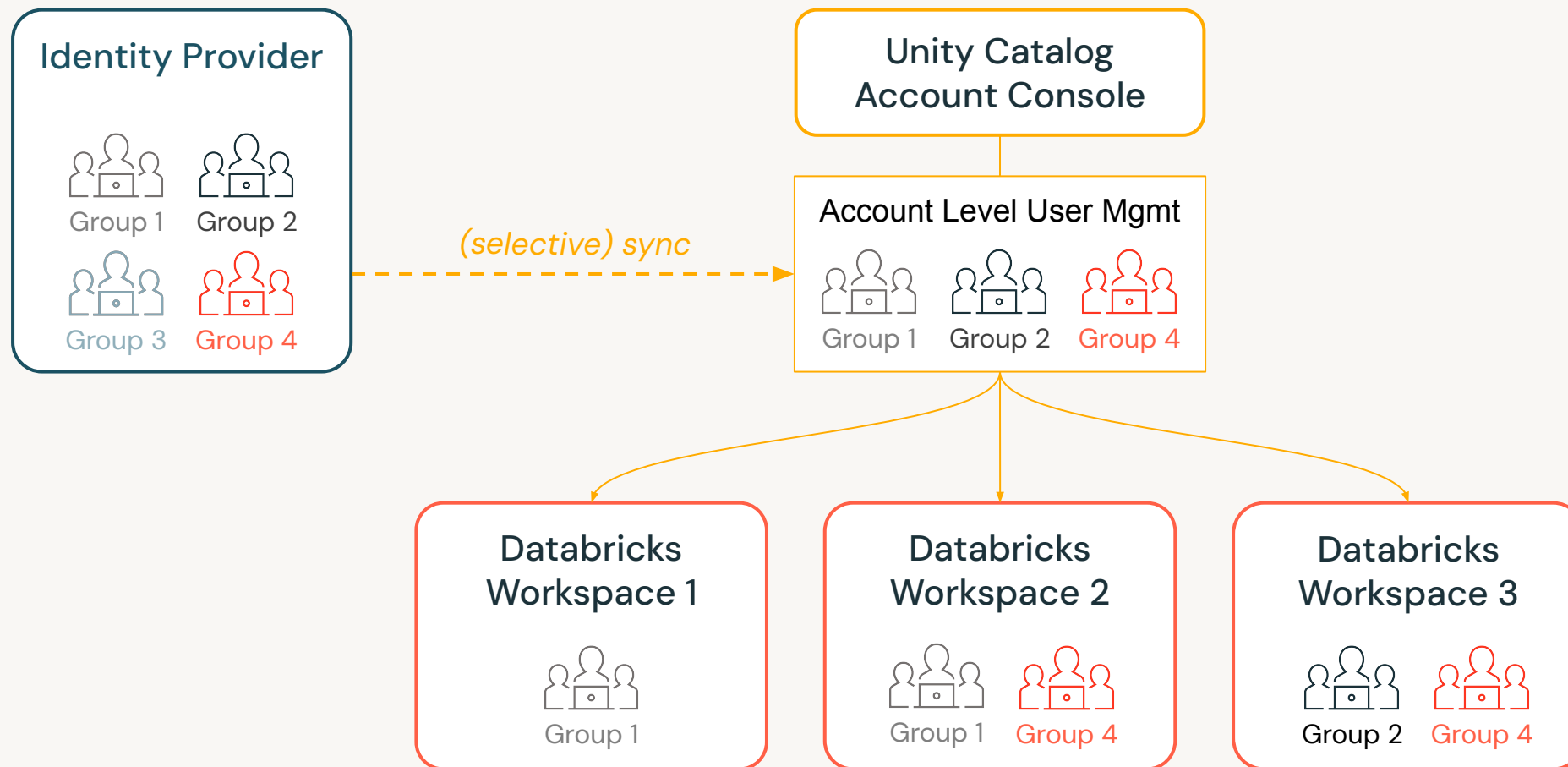
With Unity Catalog



Unity Catalog –Architecture



Identity Federation with Unity Catalog



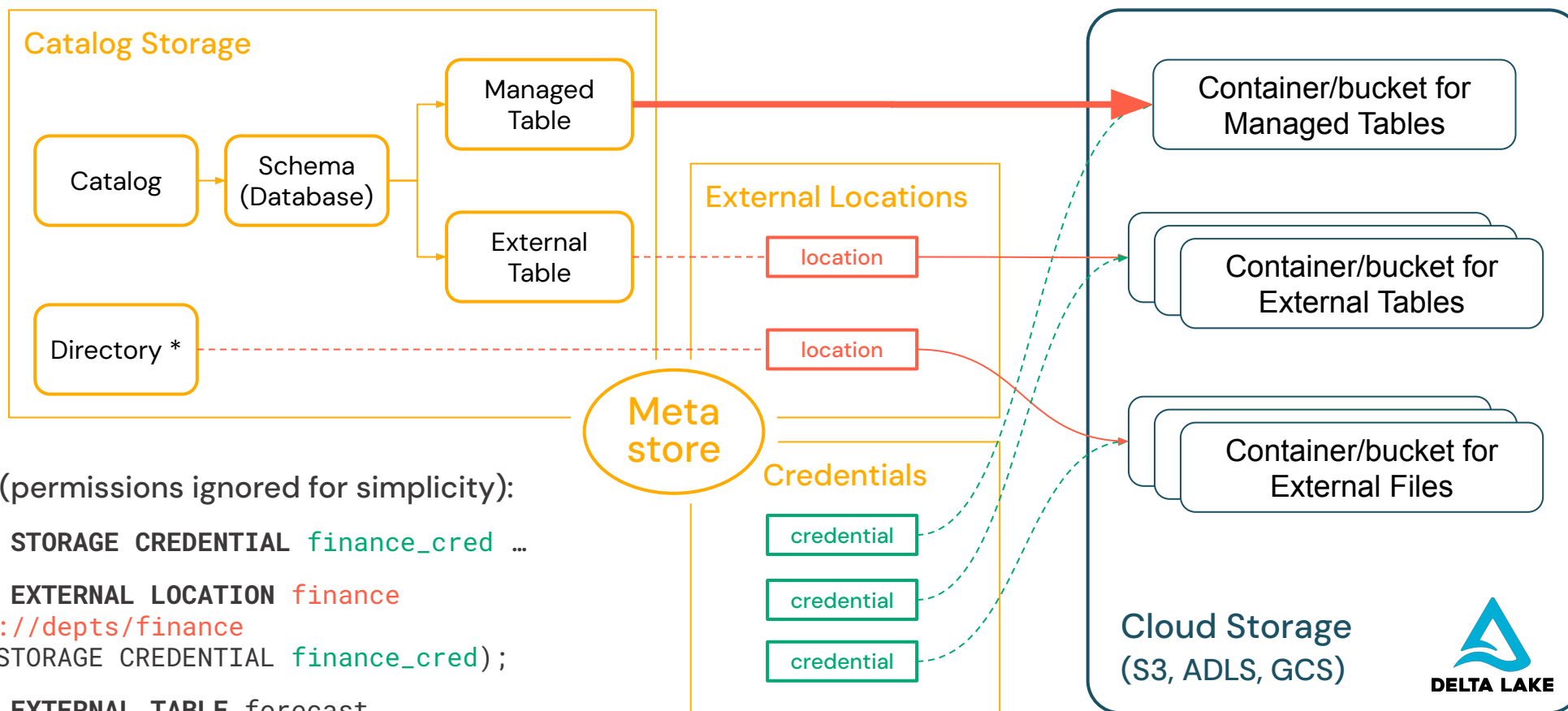
Comparison of Managed and Unmanaged UC Tables

Consider the benefits of Managed tables

Characteristic	Managed	Unmanaged (a.k.a. “External”)
Data Location	Whichever is found first: • Location specified for the database (if specified) • Location specified for the catalog (if specified) • Metastore default managed storage location	The data file specified by the LOCATION keyword in your create table statement
Performance Optimizations	Auto Tune (Liquid Clustering, etc)	Manually managed by the customer
Data Format Support	DELTA	DELTA, CSV, JSON, AVRO, PARQUET, ORC, TEXT
Value for Table Type Property	“MANAGED”	“EXTERNAL”
Create Table Syntax	<pre>CREATE TABLE [<catalog>.] [<schema>.]<table> (<column_specification>;</pre>	<pre>CREATE TABLE [<catalog>.] [<schema>.]<table> (<column_specification> LOCATION '<cloud_specific_storage_path>' ;</pre>

Overall TCO: Databricks recommends managed tables because they reduce storage and compute costs compared to external tables by learning from read/write patterns and optimizing accordingly.

Metastore, external locations and credentials



Example (permissions ignored for simplicity):

```
CREATE STORAGE CREDENTIAL finance_cred ...
```

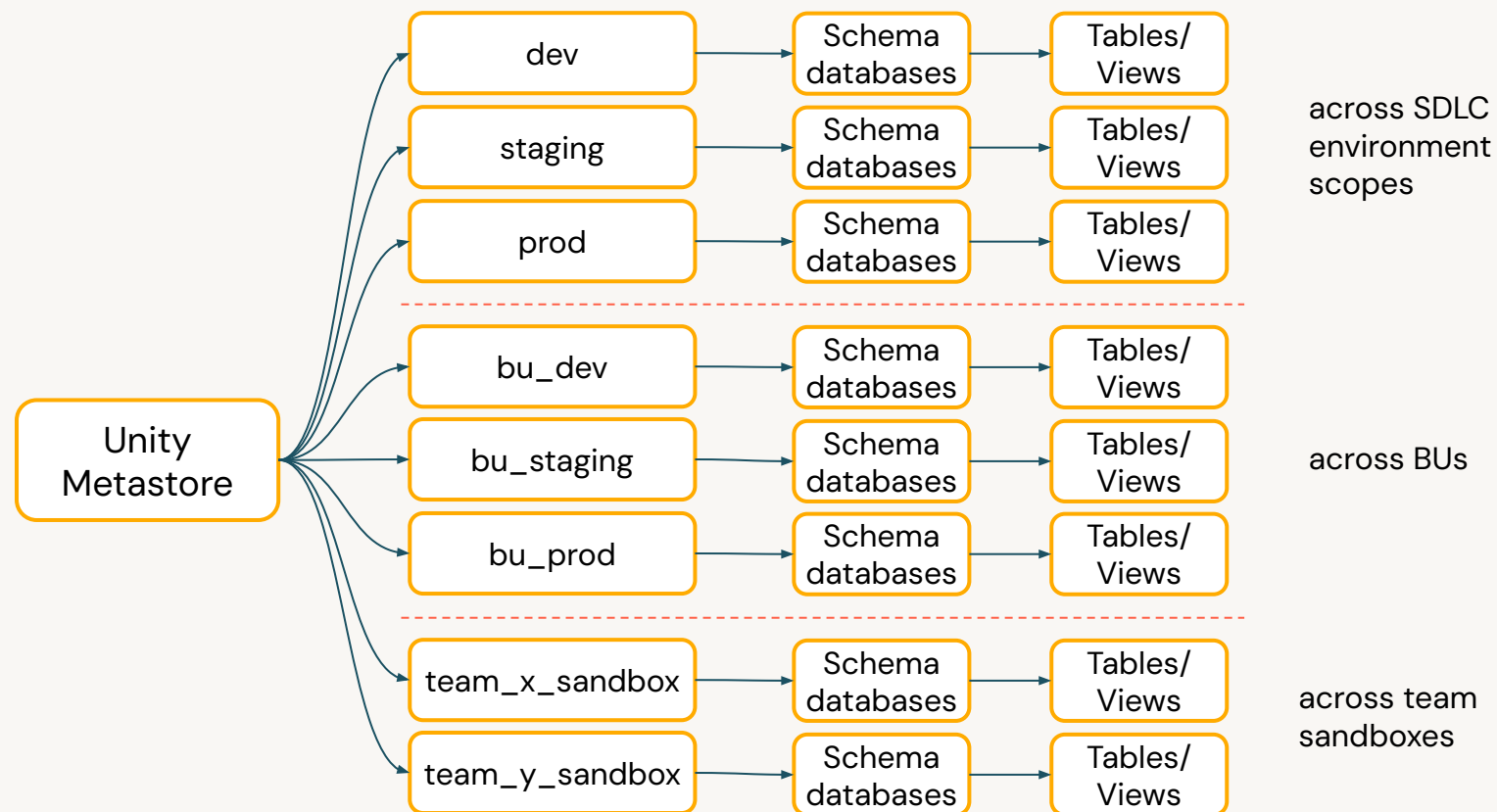
```
CREATE EXTERNAL LOCATION finance  
URL s3://depts/finance  
WITH (STORAGE CREDENTIAL finance_cred);
```

```
CREATE EXTERNAL TABLE forecast  
LOCATION s3://depts/finance/forecast;
```

```
CREATE VOLUME eu_invoices  
LOCATION s3://depts/finance/eu/invoices;
```

Suggested metastore structure

The catalog level of the 3-level namespace allows to structure databases and tables / views according to technical or business needs.



BEST PRACTICE

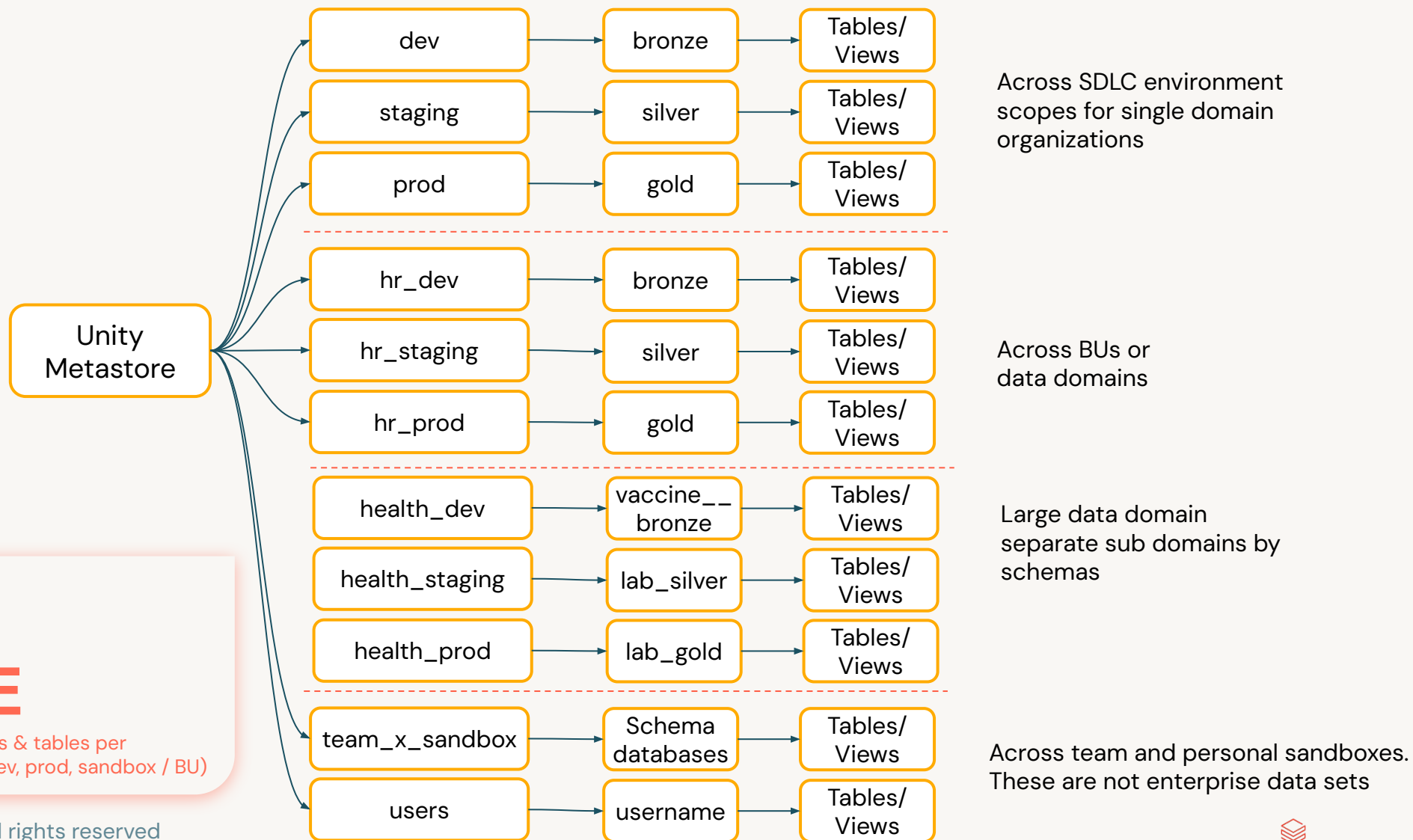
Use catalogs to structure schemas & tables per business and SDLC scopes (e.g. dev, prod, sandbox / BU)

*Catalog+Schema owned by central team.
Usage Grants performed by central team*
`GRANT USAGE on <catalog>`
`GRANT USAGE, CREATE on <schema>`

*Tables owned by team. Grants performed by teams X/Y.
Teams X, Y cannot share outside of team*



Sample metastore structure

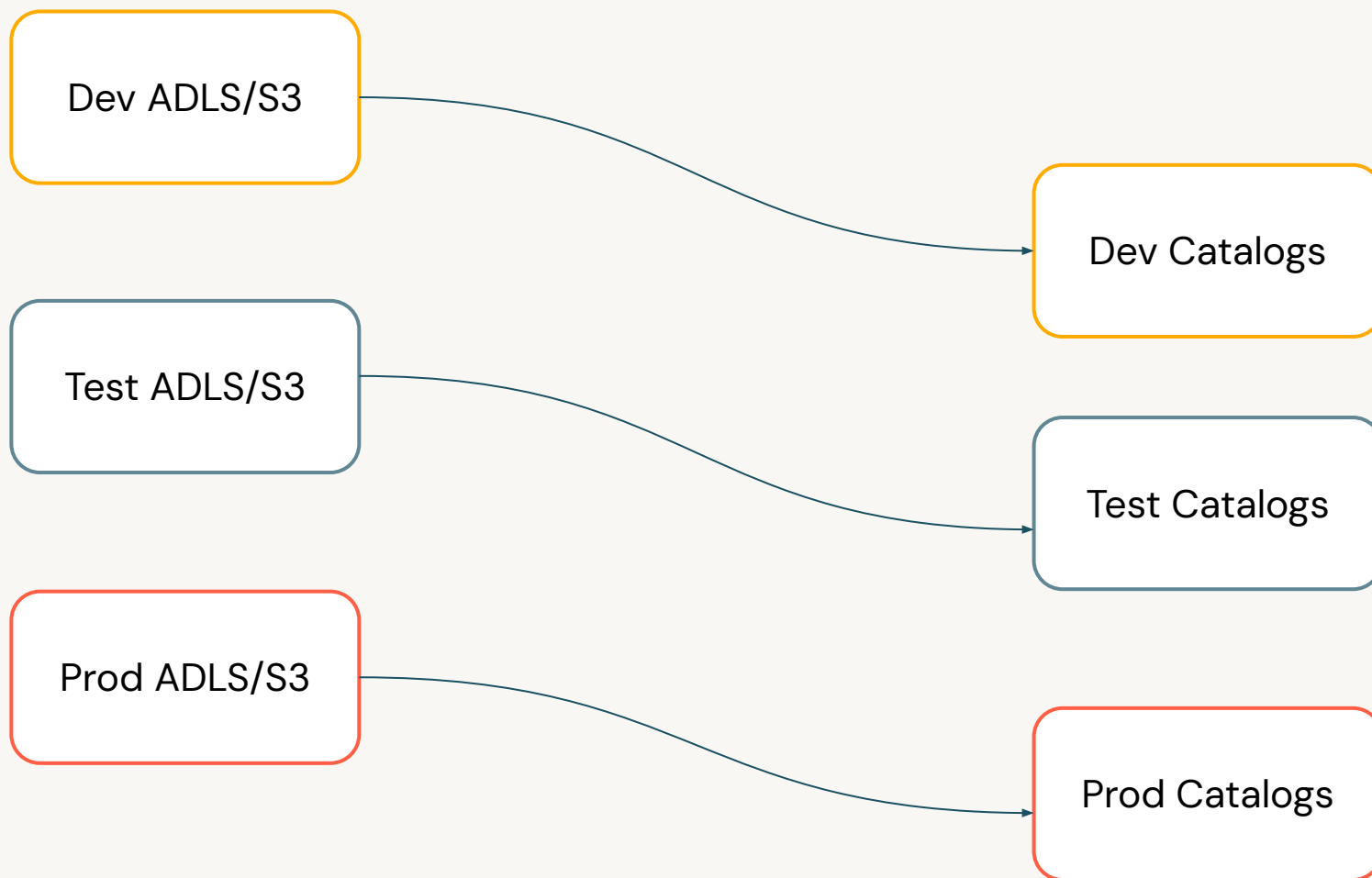


BEST PRACTICE

Use catalogs to structure schemas & tables per business and SDLC scopes (e.g. dev, prod, sandbox / BU)



Catalog Storage Mapping



Best practices

Clusters

- Use **Cluster Policies** to enforce usage of Unity-enabled clusters and latest DBR LTS
- Or Use Serverless & Skip Cluster Policies

Clusters / New Compute [UI Preview](#) [Provide feedback](#)

Unity Catalog Tech Session Cluster

Policy 

UC Demo 

☒ Multi node ☐ Single node

Access mode  **Single user access** 

Single user  Rania Benman 

Performance

Databricks runtime version 

Runtime: 11.1 (Scala 2.12, Spark 3.3.0) 

☒ Use Photon Acceleration 



Option 1: Centralized Publishing

Single cloud-region

Data Production

- Central Ingest & ETL by the central BU
- Other BUs create (business) data sets

Data Publishing

- Central BU publishes data in central PRD storage and into the PRD catalog of the Central BU in UC
- BUs requests from Central team to publish from their PRD storage to central PRD storage and into the BU catalog in UC that is maintained by Central

Data Governance (centralized)

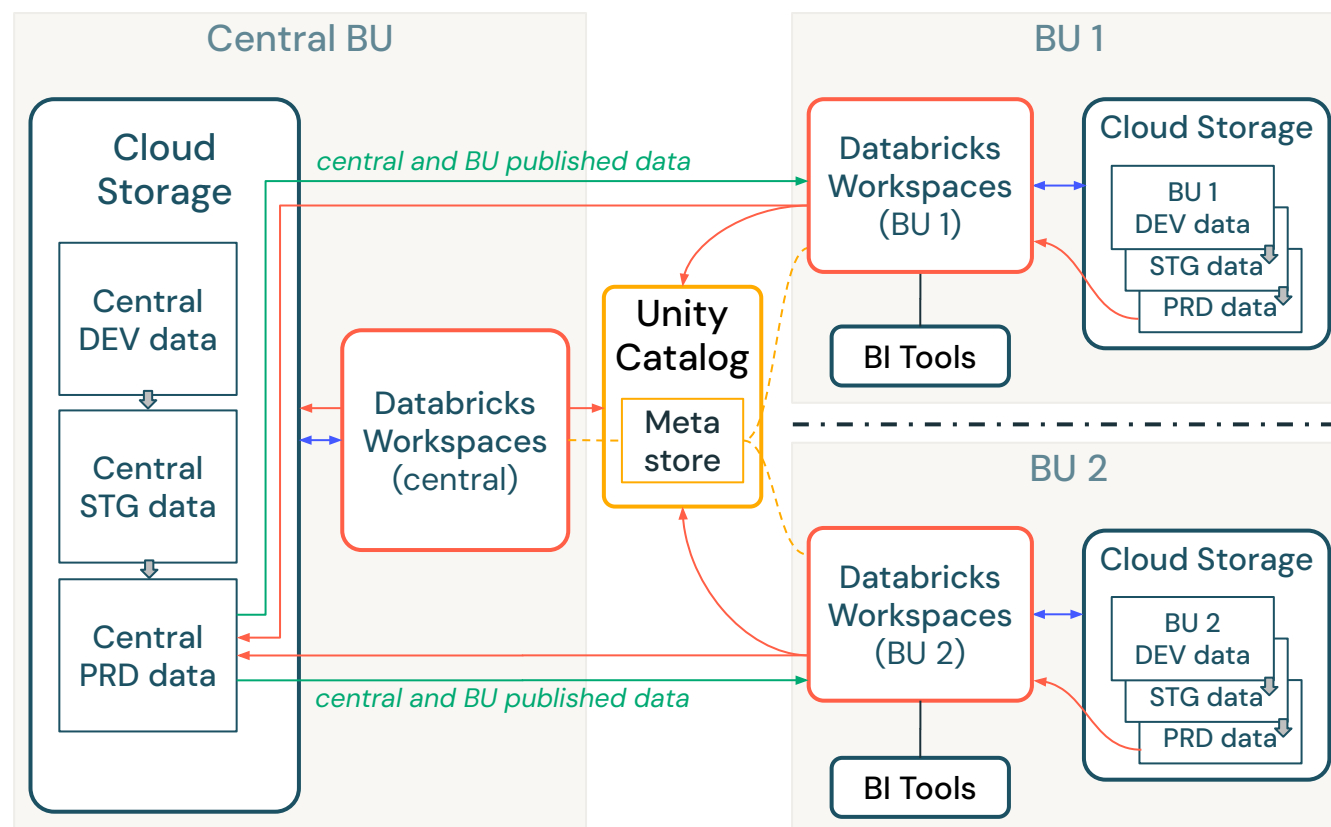
- Central team and each BU (for non published) data can work independently on their catalogs
- Central team applies additional quality assurance and maintains ACLs in the central BU catalog

Data Consumption

- Published data will be discovered in the Central catalog and consumed from the central PRD storage

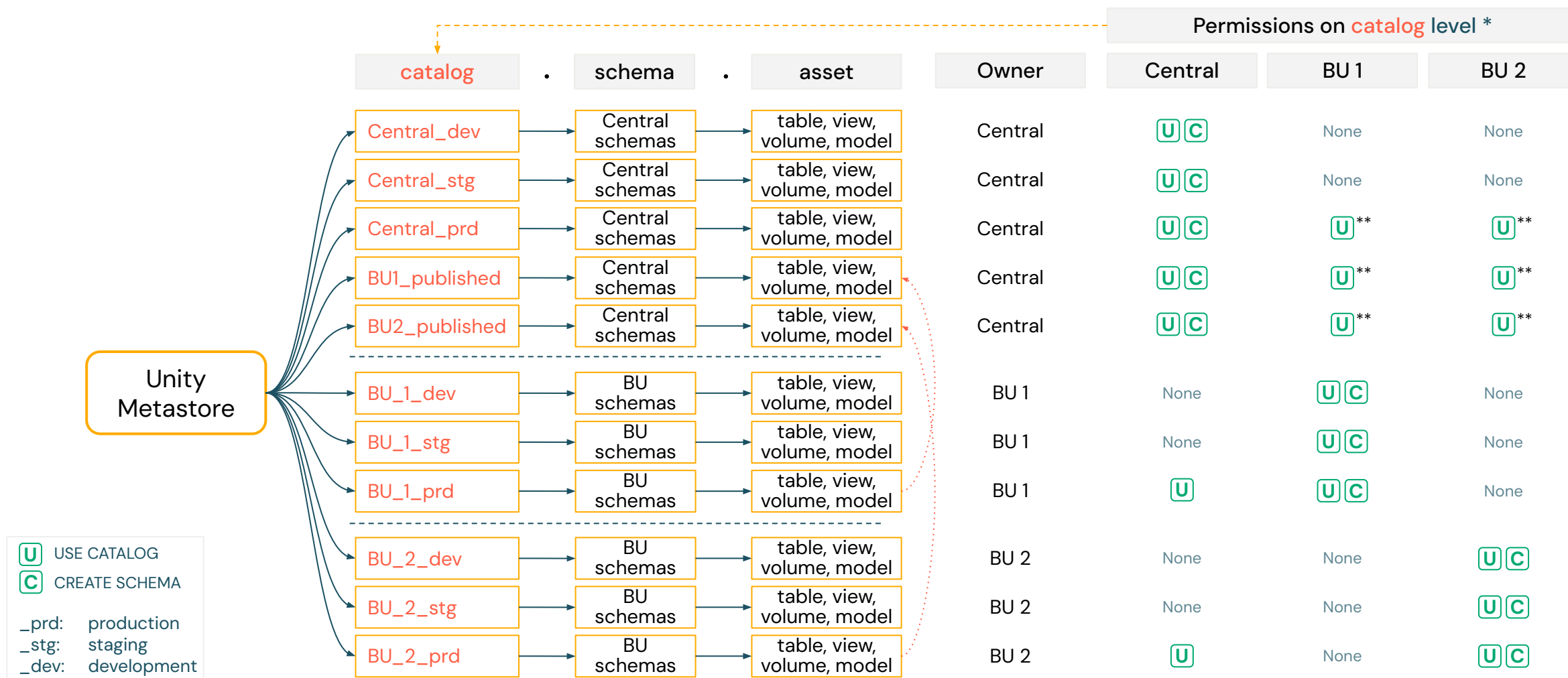
Platform operations

- Central team provides platform blueprints, creates environments for BUs (automated)
- Central team could provide common data services



- Publish data sets / tables and configure access permissions
- Consume data from other BUs
- ↔ Create and consume own databases/tables
- - - Access metadata
- - - Isolated, e.g. by network access restrictions

Option 1: Metadata Management



* To access tables, also provide USE SCHEMA and SELECT on schema and table level

To access volumes provide READ VOLUME and for models provide EXECUTE

** Catalogs where other BUs can be given access to schemas and tables/views by Central team

Option 2: Distributed Publishing

Single cloud-region

Data Production

- Central Ingest & ETL by the central BU
- Other BUs create (business) data sets

Data Publishing

- Central BU publishes data to the central PRD storage and into the catalog of the Central BU in UC
- BUs publish to their PRD storage and into BU catalogs of UC (see metadata mgmt later)

Data Governance (distributed)

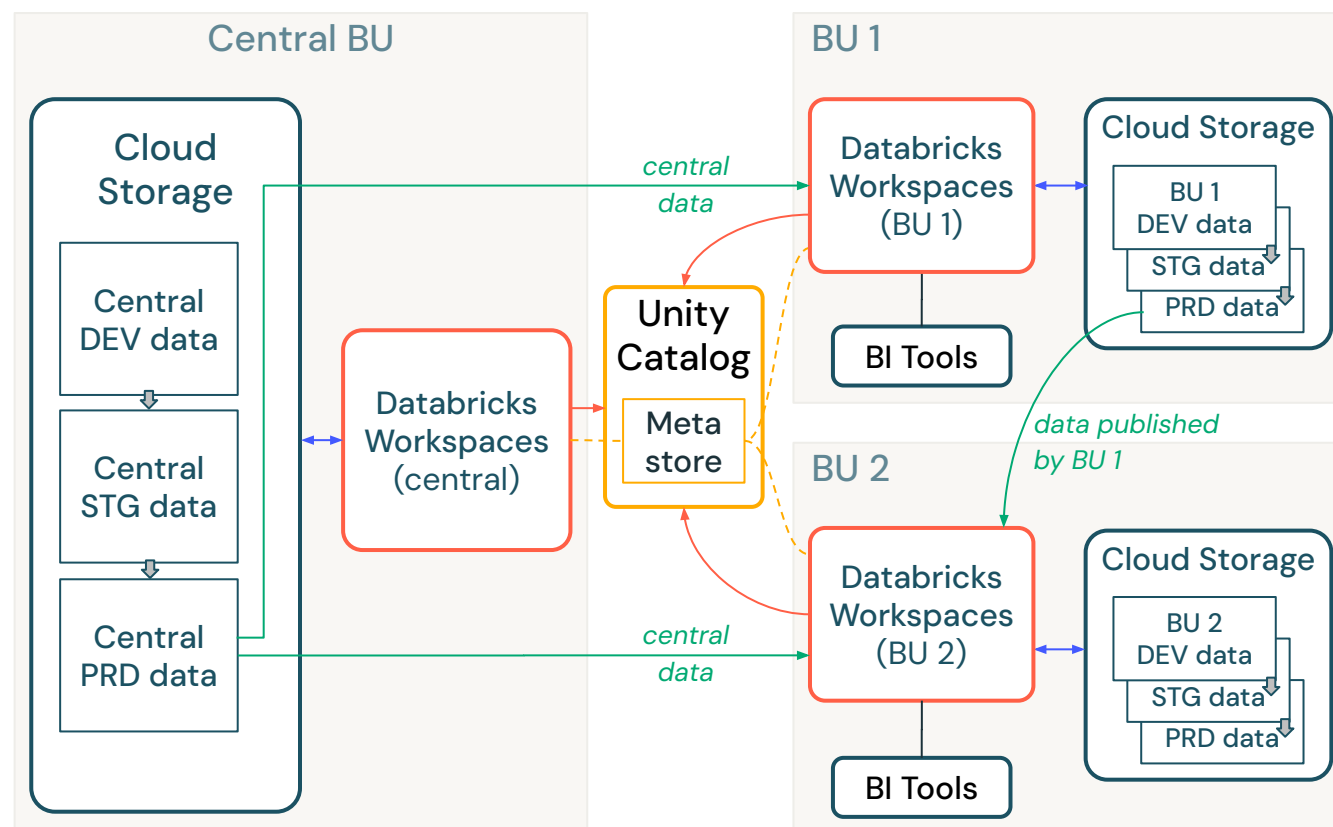
- Central team and each BU can work independently on their catalogs (publishing and setting ACLs)

Data Consumption

- Published data will be discovered in the metastore and consumed from PRD storage of the BUs (central or other)

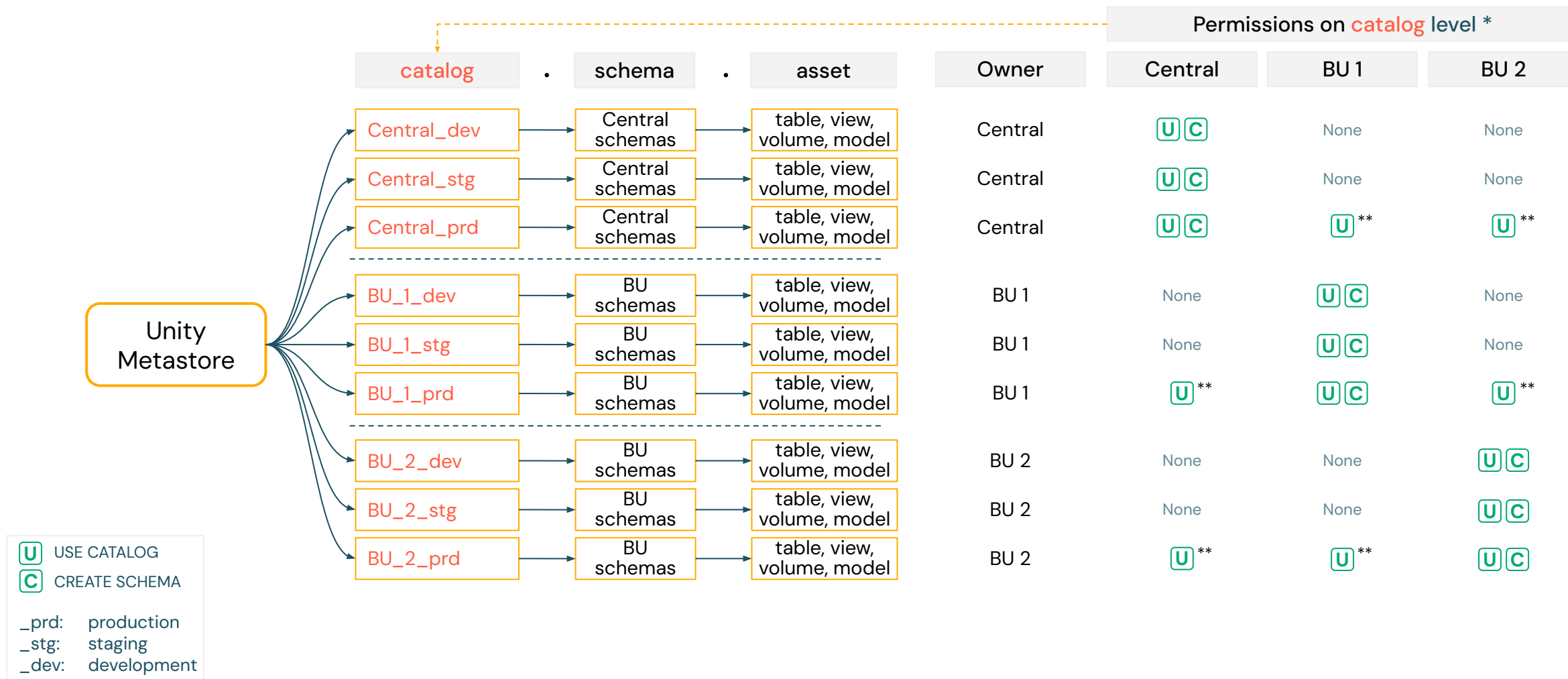
Platform operations

- Central team provides platform blueprints and creates environments for BUs (automated)



- Publish tables by configuring access permissions
- Consume other data
- ↔ Create and consume own databases/tables
- - - Access metadata

Option 2: Metadata Management



* To access tables, also provide USE SCHEMA and SELECT on schema and table level

To access volumes provide READ VOLUME and for models provide EXECUTE

** Catalogs where other BUs can be given access to schemas and tables/views by Central team

